JobKit - Tosca / SAP automation interview questions

107 questions for Tosca / SAP automation. Each answer is five pointers with working code. Single-file download.

1. What is Tosca?

1. Definition you should say first: Tricentis Tosca: model-based test automation, largely scriptless. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Tricentis Tosca is commercial model-based test automation. You scan the app into modules and reuse them; you do not start from raw Selenium code.
3. Story for interviews: Commander + modules + TCD + execution lists + DEX. Mention SAP GUI engine if that is your project.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Workspace: Customer.tws
Engines: HTML, SAP, API, Database
Run: ScratchBook locally, ExecutionList on DEX, TC-Shell in Jenkins
```

5. Calling Tosca 'scriptless so anyone can test' without naming ActionMode and identification is a shallow answer.

2. What is model-based testing in Tosca?

1. Definition you should say first: You scan the app into modules and reuse them in test cases. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. The model is the module: technical identification plus business names. Tests steer the model, so UI changes are fixed once in the module.
3. When a button id changes, rescan that attribute, not 40 test cases. Keep business names stable.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: SAP_VA01_Header
  Attribute: OrderType     Tech: GuiComboBox id=...   Business: Order Type
TestCase: Create standard order
  SAP_VA01_Header.OrderType   Input   OR
```

5. Copy-pasting modules per test destroys the model. One screen, one module family.

3. What is Tosca Commander?

1. Definition you should say first: The main UI to design, run, and manage tests. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Commander is the IDE. A workspace points at a repository. Multi-user uses a DB (check-in/out). Single-user is a local file repo.
3. Enterprises run multi-user. Check out the smallest folder, change, check in with a comment. Do not keep Commander open over a VPN sleep.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Checkout: TestCases/SAP/Order
Checkin comment: TKT-1821 WaitOn busy indicator on VA01 save
Subset export: SAP_Order_2026_09.tsu
```

5. Lost checkouts block the team. Admins recover ownership; do not clone a second workspace and edit the same path.

4. What is a workspace?

1. Definition you should say first: A Tosca repository you open in Commander. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Commander is the IDE. A workspace points at a repository. Multi-user uses a DB (check-in/out). Single-user is a local file repo.
3. Enterprises run multi-user. Check out the smallest folder, change, check in with a comment. Do not keep Commander open over a VPN sleep.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Checkout: TestCases/SAP/Order
Checkin comment: TKT-1821 WaitOn busy indicator on VA01 save
Subset export: SAP_Order_2026_09.tsu
```

5. Lost checkouts block the team. Admins recover ownership; do not clone a second workspace and edit the same path.

5. Multi-user vs single-user repo?

1. Definition you should say first: Multi-user uses a DB for check-in; single-user is file-based. Then spend the rest of the

answer on architecture, a working example, and a production failure.
2. Commander is the IDE. A workspace points at a repository. Multi-user uses a DB (check-in/out). Single-user is a local file repo.
3. Enterprises run multi-user. Check out the smallest folder, change, check in with a comment. Do not keep Commander open over a VPN sleep.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Checkout: TestCases/SAP/Order
Checkin comment: TKT-1821 WaitOn busy indicator on VA01 save
Subset export: SAP_Order_2026_09.tsu
```

5. Lost checkouts block the team. Admins recover ownership; do not clone a second workspace and edit the same path.

6. What is a module?

1. Definition you should say first: A reusable model of UI or API controls. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.
3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

7. What is XScan?

1. Definition you should say first: Scanner that identifies controls and builds modules. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.
3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

8. What is TBox?

1. Definition you should say first: Tosca's engine layer for steering applications. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.
3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

9. What is a test case?

1. Definition you should say first: A sequence of test steps using modules. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A test case is an ordered list of module steps. Each cell is a value plus ActionMode: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert.
3. Keep one business action per folder. Put data in TCD or test configuration parameters, not hardcoded passwords.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
User.Name          Input      {CP[User]}
User.Password      Input      ****
User.Login         Input      X
Home.Title         Verify     Dashboard
Home.EmpId         Buffer     EmpId
Grid.Row           Constraint Status==Open
Grid.Status        WaitOn     Open
```

5. Wrong ActionMode is the #1 Tosca bug: Input on a label, Verify without WaitOn, Buffer overwriting a global name.

10. What is a test step value?

1. Definition you should say first: The action and data on a module attribute. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A test case is an ordered list of module steps. Each cell is a value plus ActionMode: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert.

3. Keep one business action per folder. Put data in TCD or test configuration parameters, not hardcoded passwords.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
User.Name          Input      {CP[User]}
User.Password      Input      ****
User.Login         Input      X
Home.Title         Verify     Dashboard
Home.EmpId         Buffer     EmpId
Grid.Row           Constraint Status==Open
Grid.Status        WaitOn     Open
```

5. Wrong ActionMode is the #1 Tosca bug: Input on a label, Verify without WaitOn, Buffer overwriting a global name.

11. What is ActionMode?

1. Definition you should say first: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert, etc. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A test case is an ordered list of module steps. Each cell is a value plus ActionMode: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert.

3. Keep one business action per folder. Put data in TCD or test configuration parameters, not hardcoded passwords.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
User.Name          Input      {CP[User]}
User.Password      Input      ****
User.Login         Input      X
Home.Title         Verify     Dashboard
Home.EmpId         Buffer     EmpId
Grid.Row           Constraint Status==Open
Grid.Status        WaitOn     Open
```

5. Wrong ActionMode is the #1 Tosca bug: Input on a label, Verify without WaitOn, Buffer overwriting a global name.

12. What is Buffer?

1. Definition you should say first: Save a runtime value to reuse later. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.

3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number       Buffer     OrderId
Search.Order       Input      {B[OrderId]}
API.Body           Input      {"id":"{B[OrderId]}"}
Cleanup: Buffer    OrderId    <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

13. What is Verify?

1. Definition you should say first: Assert that a control equals an expected value. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

14. What is WaitOn?

1. Definition you should say first: Wait until a control reaches a value or exists. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

15. What is Constraint?

1. Definition you should say first: Limit which row or item is used. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

16. What is a TestCaseDesign sheet?

1. Definition you should say first: Data-driven instances from a template. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

17. What is an instance?

1. Definition you should say first: One data combination generated from TCD. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
```

```
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

18. What is a template test case?

1. Definition you should say first: A test case that instantiates from TCD. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

19. What is a business parameter?

1. Definition you should say first: A named value passed into a test case. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

20. What is a library in Tosca?

1. Definition you should say first: Reusable test step folders. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

21. What is a TestStepFolder?

1. Definition you should say first: A grouping of steps, sometimes used as a reusable block. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

22. What is a recovery scenario?

1. Definition you should say first: Cleanup or retry after a failed step. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

23. What is a cleanup folder?

1. Definition you should say first: Steps that run after a test to leave the app stable. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

24. What is an execution list?

1. Definition you should say first: A set of test cases to run in order. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.

3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

25. What is ScratchBook?

1. Definition you should say first: Ad-hoc run without saving results to an execution list. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.

3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

26. What is TC-Shell?

1. Definition you should say first: Command-line Tosca for CI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.

3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

27. What is DEX?

1. Definition you should say first: Distributed Execution: run tests on agent machines. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DEX is Distributed Execution: a queue, agents, and Tosca Server. AOS is the Automation Object Service in newer versions.

3. One agent per machine, same SAP GUI / browser as the project. Monitor agent heartbeat. Do not RDP-lock the session if SAP needs UI.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Tosca Server URL: https://tosca.company.local
Agent: QA-DEX-01  (Win + SAP GUI 7.70 + Chrome)
Event: ExecutionList Regression_Nightly at 22:30 IST
```

5. A disconnected RDP session kills SAP GUI tests. Use a service account autologon strategy approved by IT.

28. What is a DEX agent?

1. Definition you should say first: A machine that executes tests from a queue. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DEX is Distributed Execution: a queue, agents, and Tosca Server. AOS is the Automation Object Service in newer versions.

3. One agent per machine, same SAP GUI / browser as the project. Monitor agent heartbeat. Do not RDP-lock the session if SAP needs UI.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Tosca Server URL: https://tosca.company.local
Agent: QA-DEX-01  (Win + SAP GUI 7.70 + Chrome)
Event: ExecutionList Regression_Nightly at 22:30 IST
```

5. A disconnected RDP session kills SAP GUI tests. Use a service account autologon strategy approved by IT.

29. What is Tosca Server?

1. Definition you should say first: Services for DEX, dashboards, and integrations. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tricentis Tosca is commercial model-based test automation. You scan the app into modules and reuse them; you do not start from raw Selenium code.

3. Story for interviews: Commander + modules + TCD + execution lists + DEX. Mention SAP GUI engine if that is your project.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Workspace: Customer.tws
Engines: HTML, SAP, API, Database
Run: ScratchBook locally, ExecutionList on DEX, TC-Shell in Jenkins
```

5. Calling Tosca 'scriptless so anyone can test' without naming ActionMode and identification is a shallow answer.

30. What is AOS?

1. Definition you should say first: Automation Object Service, depending on the Tosca version. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DEX is Distributed Execution: a queue, agents, and Tosca Server. AOS is the Automation Object Service in newer versions.

3. One agent per machine, same SAP GUI / browser as the project. Monitor agent heartbeat. Do not RDP-lock the session if SAP needs UI.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Tosca Server URL: https://tosca.company.local
Agent: QA-DEX-01  (Win + SAP GUI 7.70 + Chrome)
Event: ExecutionList Regression_Nightly at 22:30 IST
```

5. A disconnected RDP session kills SAP GUI tests. Use a service account autologon strategy approved by IT.

31. What is TDS?

1. Definition you should say first: Test Data Service: managed test data. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TDS / Test Data Service provisions and masks data so tests do not steal each other's orders.

3. Create a pool of customers. Checkout a record in the test, check in after cleanup. Never hardcode a prod customer.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TDS type: Customer
  Status: Available -> InUse -> Consumed
Test: checkout Customer where Country==IN
Buffer CustId
Cleanup: set Available if the order rolled back
```

5. A shared QA client 1000 used by 12 testers will make Verify fail randomly.

32. What is Test Data Management?

1. Definition you should say first: Create, mask, and provision data for tests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TDS / Test Data Service provisions and masks data so tests do not steal each other's orders.

3. Create a pool of customers. Checkout a record in the test, check in after cleanup. Never hardcode a prod customer.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TDS type: Customer
  Status: Available -> InUse -> Consumed
Test: checkout Customer where Country==IN
Buffer CustId
Cleanup: set Available if the order rolled back
```

5. A shared QA client 1000 used by 12 testers will make Verify fail randomly.

33. How do you scan a web page?

1. Definition you should say first: XScan with the browser extension or engine, then map controls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

34. What is a steering engine?

1. Definition you should say first: The adapter for HTML, SAP, API, and others. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

35. What is the HTML engine?

1. Definition you should say first: Steers browser DOM controls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen

changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

36. What is the SAP engine?

1. Definition you should say first: Steers SAP GUI controls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

37. What is API Scan?

1. Definition you should say first: Build modules from REST/SOAP definitions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

38. How do you test REST in Tosca?

1. Definition you should say first: API Scan or API Engine modules with payload and verify. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

39. What is a payload?

1. Definition you should say first: JSON or XML body you send. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

40. How do you verify status 200?

1. Definition you should say first: Verify the status attribute on the API module. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

41. How do you chain API calls?

1. Definition you should say first: Buffer an id from the first response into the second request. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

42. What is a module attribute?

1. Definition you should say first: One field or control in a module. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

43. What is a technical ID?

1. Definition you should say first: How Tosca finds the control, for example id or xpath-like path. Then spend the rest of the

answer on architecture, a working example, and a production failure.
2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.
3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

44. What if XScan cannot find a control?

1. Definition you should say first: Try another engine, frame, or identify by parent. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.
3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

45. What is a dynamic ID problem?

1. Definition you should say first: IDs change each session; use a stable property. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.
3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

46. How do you handle iFrames?

1. Definition you should say first: Scan inside the frame or switch context per engine rules. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.
3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

47. How do you handle a dropdown?

1. Definition you should say first: Input or Select action on the combo control. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

48. How do you handle a table?

1. Definition you should say first: Constraint on a row plus Input/Verify on cells. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.
3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

49. How do you upload a file?

1. Definition you should say first: File control Input with a path the agent can see. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.
3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

50. How do you handle CAPTCHA?

1. Definition you should say first: You do not automate CAPTCHA; tests skip or use a test hook. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.
3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

51. How do you handle SSO?

1. Definition you should say first: Test account plus storage of a session if the project allows. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.
3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

52. What is a business-focused test case?

1. Definition you should say first: Steps named in business language, modules underneath. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

53. What is risk-based testing in Tosca?

1. Definition you should say first: Weight tests by business risk if the project uses it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

54. What is exploratory testing vs Tosca?

1. Definition you should say first: Exploratory is unscripted; Tosca is designed automation. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

55. What is a requirements folder?

1. Definition you should say first: Optional mapping of tests to requirements. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

56. What is coverage in Tosca?

1. Definition you should say first: Which requirements or TCD instances ran. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted

and complements Tosca. Requirements folders map coverage.
3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

57. How do you run from CI?

1. Definition you should say first: TC-Shell or Tosca CI against a workspace and execution list. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.
3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

58. How do you pass results to Jenkins?

1. Definition you should say first: CI result files or Tosca Server reporting. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.
3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

59. What is a configuration parameter?

1. Definition you should say first: Settings such as browser, timeout, or test config. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.
3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

60. What is TCP?

1. Definition you should say first: Test configuration parameter. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.
3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
```

```
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

61. How do you run Chrome vs Edge?

1. Definition you should say first: TCP or test configuration for the browser. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.

3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

62. What is a buffer syntax?

1. Definition you should say first: Curly braces with the buffer name, depending on version. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.

3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number        Buffer     OrderId
Search.Order        Input      {B[OrderId]}
API.Body            Input      {"id":"{B[OrderId]}"}
Cleanup: Buffer     OrderId    <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

63. What is a date expression in Tosca?

1. Definition you should say first: Dynamic date functions in test step values. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca date expressions compute today+N. Random values need a stored seed if the value must be reused or cleaned up.

3. Use {DATE[yyyyMMdd][+1]} style expressions per your version. Buffer random keys immediately.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ValidFrom     Input   {DATE[dd.MM.yyyy]}
ValidTo       Input   {DATE[dd.MM.yyyy][+30]}
PoNumber      Input   PO{RANDOM[6]}
PoNumber      Buffer  PoNumber
```

5. Random emails without a unique domain will collide and poison the environment.

64. What is a random value?

1. Definition you should say first: Generated test data; do not use it for unique keys without control. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca date expressions compute today+N. Random values need a stored seed if the value must be reused or cleaned up.

3. Use {DATE[yyyyMMdd][+1]} style expressions per your version. Buffer random keys immediately.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ValidFrom     Input   {DATE[dd.MM.yyyy]}
ValidTo       Input   {DATE[dd.MM.yyyy][+30]}
PoNumber      Input   PO{RANDOM[6]}
PoNumber      Buffer  PoNumber
```

5. Random emails without a unique domain will collide and poison the environment.

65. How do you wait for a spinner?

1. Definition you should say first: WaitOn existence or wait for the spinner to vanish. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

66. Why not use a huge static wait?

1. Definition you should say first: It slows suites and still flakes; WaitOn is better. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

67. What is synchronization?

1. Definition you should say first: Waiting for the app to be ready before the next step. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

68. What is a flaky test?

1. Definition you should say first: Passes and fails without product bugs; fix waits and identifications. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

69. How do you stabilize SAP tests?

1. Definition you should say first: SAP engine, proper GUI scripting, and wait for busy indicators. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode           Input     /nVA01
```

```
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

70. What is SAP scripting setting?

1. Definition you should say first: SAP GUI must allow scripting on the client and server. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode          Input     /nVA01
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

71. What is a TCode?

1. Definition you should say first: SAP transaction code you steer to. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode          Input     /nVA01
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

72. How do you test an SAP Fiori app?

1. Definition you should say first: Often HTML/browser engine, not classic SAP GUI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode          Input     /nVA01
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

73. What is a Tosca license type?

1. Definition you should say first: Commander, execution-only, etc. - follow what the project bought. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

74. What is a subset?

1. Definition you should say first: An export of part of the repo to share. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).
3. Small folders, daily check-in, one owner per module family.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

75. How do you avoid merge conflicts?

1. Definition you should say first: Small check-outs, folders by team, frequent check-in. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Tosca is model-based: XScan builds modules, test cases steer them, TCD supplies data, DEX runs agents.
3. Name ActionMode, buffers, WaitOn, and how CI calls TC-Shell. Never demo a 30-second static wait.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Login.User          Input     {CP[User]}
Login.Password      Input     {PW}
Login.SignIn        Input     X
Home.Welcome        Verify    Welcome*
Home.OrderId        Buffer    OrderId
```

5. Flakes come from dynamic IDs, missing SAP scripting, shared buffers, and no recovery after a leftover dialog.

76. What is a checkout?

1. Definition you should say first: Lock objects so you can edit in multi-user. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).
3. Small folders, daily check-in, one owner per module family.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

77. What is a check-in?

1. Definition you should say first: Save objects back to the common repo. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).
3. Small folders, daily check-in, one owner per module family.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

78. What happens if you lose a checkout?

1. Definition you should say first: Admin recover or take ownership per process. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

79. What is a branch in Tosca?

1. Definition you should say first: Versioning approach depends on repo type and version. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).
3. Small folders, daily check-in, one owner per module family.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

80. How do you review a test case?

1. Definition you should say first: Peer review modules, data, and verifies, not only a green run. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.
3. DoD: two green DEX runs, reviewed, data source documented, recovery present.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

81. What is a smoke execution list?

1. Definition you should say first: A short list that proves the app starts and login works. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.
3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

82. What is a regression pack?

1. Definition you should say first: The suite you run before a release. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.
3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

83. How do you select tests for a change?

JobKit - Tosca / SAP automation interview questions

1. Definition you should say first: Impact on modules used by the changed screens. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.
3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

84. What is a data-driven test?

1. Definition you should say first: Same steps, many TCD instances. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.
3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

85. What is a keyword-driven test?

1. Definition you should say first: Reusable business steps; Tosca libraries play that role. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.
3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

86. Tosca vs Selenium?

1. Definition you should say first: Tosca is model-based commercial; Selenium is code-first OSS. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.
3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

87. When is Tosca a poor fit?

1. Definition you should say first: Tiny apps, heavy CAPTCHA, or teams that only want code. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.
3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

88. What is a TestCase workstate?

1. Definition you should say first: Status such as COMPLETED used in some processes. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

89. What is a log in Tosca?

1. Definition you should say first: Run log with passed/failed steps and screenshots if configured. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

90. How do you capture a screenshot on fail?

1. Definition you should say first: Execution settings for screenshots. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

91. What is a recovery scenario example?

1. Definition you should say first: Close a leftover dialog so the next case can start. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

92. How do you skip a step?

1. Definition you should say first: Condition or disable the step; do not leave broken verifies. Then spend the rest of the

answer on architecture, a working example, and a production failure.
2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.
3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

93. What is a condition in Tosca?

1. Definition you should say first: If-then style control of whether steps run. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.
3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

94. What is a repetition?

1. Definition you should say first: Loop a folder while a condition holds. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.
3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

95. How do you loop a table?

1. Definition you should say first: Repetition with Constraint moving through rows. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.
3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

96. What is a buffer overwrite risk?

1. Definition you should say first: Two tests share a buffer name and leak data. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.
3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number        Buffer     OrderId
Search.Order        Input      {B[OrderId]}
API.Body            Input      {"id":"{B[OrderId]}"}
Cleanup: Buffer     OrderId    <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

97. How do you isolate buffers?

1. Definition you should say first: Unique names per test or clear them in cleanup. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.

3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number        Buffer     OrderId
Search.Order        Input      {B[OrderId]}
API.Body            Input      {"id":"{B[OrderId]}"}
Cleanup: Buffer     OrderId    <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

98. What is a test configuration 'Standard'?

1. Definition you should say first: Default config used if none is specified. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Standard is the default TCP set when none is selected. Agents and laptops must agree on URL and browser or results lie.

3. Create QA, UAT, PROD-readonly configs. Never point Standard at production.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Config Standard: Url=https://qa.example.com Browser=Chrome
Config UAT: Url=https://uat.example.com Browser=Edge
```

5. A leftover Standard URL from a demo will smash the wrong environment.

99. How do you handle 2FA in tests?

1. Definition you should say first: Test hooks or a non-prod OTP process. Never bypass production 2FA. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

100. Should you automate production?

1. Definition you should say first: Prefer test environments; production automation is rare and gated. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

101. What is a service virtualization?

1. Definition you should say first: Fake a dependency; Tosca may call a stub instead of a live API. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

102. What is a contract test?

1. Definition you should say first: API schema and status checks between services. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

103. How do you version modules?

1. Definition you should say first: Change with the app; keep old tests green or retire them. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

104. What is technical debt in Tosca?

1. Definition you should say first: Duplicate modules, absolute waits, and unused instances. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

105. How do you name modules?

1. Definition you should say first: Screen plus control group, stable and unique. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

106. How do you name test cases?

1. Definition you should say first: Story or risk plus expected outcome. Then spend the rest of the answer on architecture, a

working example, and a production failure.
2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.
3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

107. What is a Definition of Done for a test?

1. Definition you should say first: Runs green twice on CI, reviewed, and data is documented. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.
3. DoD: two green DEX runs, reviewed, data source documented, recovery present.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.